BGP HIJACKING

BITAN Shmuel

22/01/2026

—

RSX217

—

CNAM PARIS

Contents

1. Mise en place de l'environnement et difficultés techniques 3
1.1 Contexte initial du projet 3
1.2 Problèmes de dépendances et compatibilité Python 3
1.3 Phase de débogage et recherche de versions corrigées 3
2. Fonctionnement général de BGP et principes des attaques de BGP Hijacking 4
2.1 Présentation générale de BGP 4
2.2 Les principaux attributs BGP 4
2.3 Modèle de confiance et limites de sécurité de BGP 5
2.4 Principe des attaques de BGP Hijacking 5
2.5 Hijacking par AS_PATH plus court 5
2.6 Hijacking par annonce de préfixe plus spécifique 5
2.7 Objectifs de l'étude expérimentale 6
3. Description de la topologie et fonctionnement normal du routage BGP 6
3.1 Présentation générale de la topologie 6
3.2 Rôle des différents systèmes autonomes 7
3.3 Mise en place du routage BGP normal 7
3.4 Annonces BGP dans l'état normal 7
3.5 Validation applicative du routage normal 8
4. Attaque de BGP Hijacking par annonce du même préfixe avec un AS_PATH plus court 8
4.1 Principe de l'attaque 8
4.2 Lancement de l'attaque 9
4.3 Observation des annonces BGP 9
4.4 Visibilité de l'attaque depuis AS2 10
4.5 Impact applicatif de l'attaque 10
4.6 Analyse et limites de l'attaque 11
4.7 Conclusion sur l'attaque par AS_PATH plus court 11
5. Attaque de BGP Hijacking par annonce d'un préfixe plus spécifique 11
5.1 Principe et motivation de l'attaque 11
5.2 Lancement de l'attaque 12
5.3 Observation des annonces BGP 12
5.4 Impact sur AS2 : un cas particulièrement révélateur 12
5.5 Analyse du changement de décision de routage 13
5.6 Impact applicatif de l'attaque 13
5.7 Comparaison avec l'attaque par AS_PATH plus court 14
5.8 Conclusion sur l'attaque par préfixe plus spécifique 14
Conclusion générale 14

1. Mise en place de l'environnement et difficultés techniques

1.1 Contexte initial du projet

L'objectif de ce projet était de démontrer des attaques de type **BGP Hijacking** dans un environnement contrôlé, à l'aide de l'outil **Mininet** et de routeurs BGP simulés via **FRRouting (FRR)**.
Le projet s'appuie sur un code existant, initialement conçu pour des démonstrations académiques plus anciennes, notamment autour de Mininet et de BGP.

Dès les premières phases de mise en place, plusieurs difficultés techniques sont apparues, principalement liées à l'**ancienneté du code**, à des **dépendances obsolètes**, ainsi qu'à des **incompatibilités entre Python 2 et Python 3**.

1.2 Problèmes de dépendances et compatibilité Python

Le code source original du projet avait été développé pour fonctionner avec **Python 2**, aujourd'hui obsolète et non supporté par défaut sur les systèmes récents.
Or, l'environnement de travail utilisé repose sur **Python 3**, ce qui a immédiatement entraîné plusieurs erreurs à l'exécution, notamment :

- erreurs de syntaxe liées aux instructions `print` ;
- incompatibilités dans la gestion des chaînes de caractères (bytes / unicode) ;
- problèmes avec certains modules Python non maintenus ou absents.

De plus, certaines dépendances logicielles nécessaires au bon fonctionnement du projet n'étaient plus disponibles dans leurs versions initiales, ou avaient été remplacées par des alternatives modernes.
C'est notamment le cas des outils de routage BGP, où **Quagga** (utilisé historiquement) a été remplacé par **FRRouting (FRR)**.

1.3 Phase de débogage et recherche de versions corrigées

La mise en place de l'environnement expérimental a nécessité une phase de débogage approfondie en raison de plusieurs incompatibilités entre le code initial du projet et l'environnement d'exécution actuel. Les principales difficultés rencontrées concernaient l'utilisation de Python 2, des appels système obsolètes et la gestion des processus réseau au sein de Mininet.
L'analyse des erreurs à l'exécution a permis d'identifier les portions de code incompatibles et d'approfondir la compréhension du fonctionnement interne de Mininet, notamment la gestion des *namespaces* et des processus isolés.

Afin de limiter les modifications manuelles, une étude de versions alternatives du projet a été menée à partir de différents *forks* disponibles sur GitHub. Ces implémentations plus récentes, certaines en Python 3, ont servi de base pour adapter le code existant tout en conservant la logique initiale du projet.

Les corrections apportées ont principalement consisté en l'adaptation complète du code en Python 3, la mise à jour des commandes exécutées via Mininet, ainsi que la correction des chemins et des scripts de lancement des démons de routage FRRouting. Une attention particulière a été portée à la gestion des processus BGP afin d'assurer une initialisation propre, d'éviter les conflits entre instances et de garantir une convergence correcte du routage lors des phases d'attaque et de retour à l'état nominal.
Une fois ces ajustements réalisés, la topologie a pu être lancée de manière stable à l'aide de la commande sudo python3 bgp.py.

2. Fonctionnement général de BGP et principes des attaques de BGP Hijacking

2.1 Présentation générale de BGP

Le **Border Gateway Protocol (BGP)** est le protocole de routage inter-domaine utilisé sur Internet.
Son rôle principal est de permettre l'échange d'informations de routage entre des **systèmes autonomes (Autonomous Systems – AS)**, qui représentent des réseaux administrés par des entités distinctes (fournisseurs d'accès, entreprises, institutions, etc.).

Contrairement aux protocoles de routage internes (IGP) tels que OSPF ou RIP, BGP ne cherche pas à calculer le chemin le plus court en termes de distance, mais à sélectionner le **chemin le plus approprié selon des critères de politique et d'attributs de routage**.

BGP repose sur l'établissement de sessions TCP entre routeurs voisins, utilisant le port 179. À travers ces sessions, les routeurs échangent des annonces de préfixes IP atteignables ainsi que des informations décrivant le chemin emprunté pour les atteindre. Chaque annonce contient un ensemble d'attributs qui permettent au routeur de prendre une décision de routage lorsqu'il existe plusieurs chemins possibles vers un même réseau.

2.2 Les principaux attributs BGP

Chaque annonce BGP contient plusieurs attributs permettant aux routeurs de prendre une décision de routage.
Parmi les plus importants, on peut citer :

- **AS_PATH** : liste ordonnée des systèmes autonomes traversés pour atteindre un préfixe. Un AS_PATH plus court est généralement préféré.
- **NEXT_HOP** : adresse IP du prochain routeur vers lequel le trafic doit être envoyé.
- **ORIGIN** : indique l'origine de la route (IGP, EGP ou Incomplete).
- **Préfixe IP (NLRI)** : réseau annoncé, par exemple `13.0.0.0/8`.

Lorsqu'un routeur reçoit plusieurs annonces pour un même préfixe, il applique un **processus de sélection** afin de déterminer la route à installer dans sa table de routage.

Un point fondamental du fonctionnement de BGP est la règle du **Longest Prefix Match** :

à préfixe égal, la route la plus spécifique (par exemple `/24`) est toujours préférée à une route plus générale (par exemple `/8`).

2.3 Modèle de confiance et limites de sécurité de BGP

Historiquement, BGP a été conçu dans un contexte de **confiance mutuelle entre opérateurs réseau**.
Par conséquent, le protocole **ne vérifie pas l'authenticité des annonces reçues** : un routeur BGP accepte généralement les routes annoncées par ses voisins sans mécanisme de validation cryptographique.

Cette absence de contrôle expose BGP à des vulnérabilités majeures, qu'il s'agisse d'erreurs de configuration accidentelles ou d'annonces malveillantes intentionnelles.

Ces vulnérabilités constituent le fondement des attaques de type **BGP Hijacking**.

2.4 Principe des attaques de BGP Hijacking

Une attaque de **BGP Hijacking** consiste à **annoncer frauduleusement un préfixe IP** afin de détourner le trafic réseau vers un AS malveillant ou mal configuré.

L'objectif de l'attaquant peut être multiple :

- interception du trafic (man-in-the-middle) ;
- espionnage ;
- modification des données ;
- déni de service (blackhole) ;
- usurpation de services légitimes.

Dans le cadre de ce projet, deux mécanismes principaux de hijacking sont étudiés.

2.5 Hijacking par AS_PATH plus court

Dans ce scénario, l'attaquant annonce **le même préfixe IP que le réseau légitime**, mais avec un **AS_PATH plus court**.
Étant donné que BGP privilégie les chemins contenant moins d'AS, cette annonce devient plus attractive pour les routeurs voisins.

Ainsi, même si le préfixe annoncé est identique, la différence de longueur de chemin suffit à provoquer un **changement de route** et une redirection du trafic vers l'attaquant.

Cette attaque est relativement simple à mettre en œuvre et repose directement sur le mécanisme de sélection de route de BGP.

2.6 Hijacking par annonce de préfixe plus spécifique

Un second type d'attaque repose sur la règle du **Longest Prefix Match**.
Dans ce cas, l'attaquant n'annonce pas exactement le même préfixe que le réseau légitime, mais un **sous-préfixe plus spécifique**.

Par exemple :

- réseau légitime : `13.0.0.0/8`
- attaquant : `13.0.1.0/24`

Même si le chemin annoncé par l'attaquant est plus long ou moins optimal, la route `/24` sera toujours préférée à la route `/8`.
Ce type d'attaque est particulièrement dangereux, car il est **très difficile à détecter** et peut affecter uniquement une partie ciblée du trafic.

2.7 Objectifs de l'étude expérimentale

Dans ce projet, l'objectif est de :

- comprendre les mécanismes internes de BGP ;
- illustrer deux types distincts d'attaques de BGP Hijacking ;
- observer leurs impacts sur le routage et sur les applications ;
- mettre en évidence la facilité avec laquelle le trafic peut être détourné dans un environnement non sécurisé.

Les parties suivantes détaillent la topologie utilisée, le fonctionnement normal du routage BGP, puis l'analyse détaillée de chaque attaque mise en œuvre.

3. Description de la topologie et fonctionnement normal du routage BGP

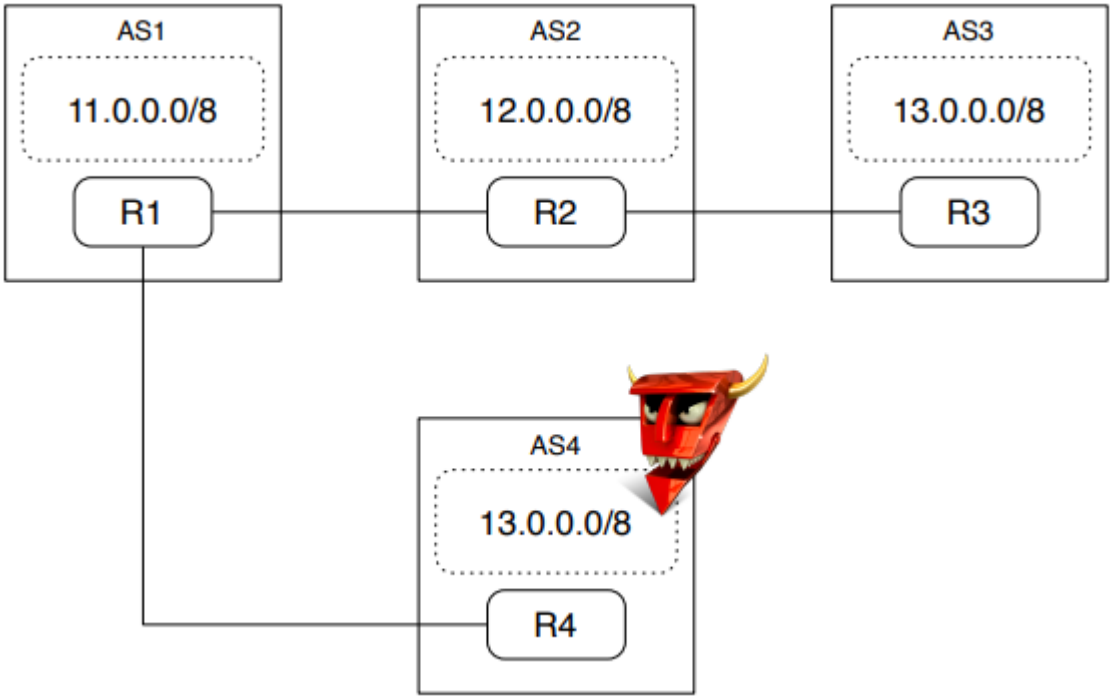
3.1 Présentation générale de la topologie

Afin d'étudier le comportement du protocole BGP dans un environnement contrôlé, une topologie réseau a été mise en place à l'aide de l'outil **Mininet**.
Cette topologie vise à reproduire un scénario simplifié mais réaliste d'échanges BGP entre plusieurs systèmes autonomes.

La topologie est composée de **quatre routeurs principaux**, chacun représentant un système autonome distinct :

- **S1 (AS1)** : routeur central jouant le rôle de point d'entrée du réseau ;
- **S2 (AS2)** : système autonome intermédiaire ;
- **S3 (AS3)** : système autonome hébergeant le serveur légitime ;
- **S4 (AS4)** : système autonome utilisé ultérieurement comme routeur attaquant.

Chaque système autonome est relié à plusieurs hôtes, simulant des réseaux clients, et les routeurs sont interconnectés afin de permettre l'échange d'annonces BGP.



3.2 Rôle des différents systèmes autonomes

Dans l'état normal du réseau (sans attaque), chaque AS joue un rôle bien défini :

- **AS3 (S3)** annonce le préfixe IP légitime `13.0.0.0/8`, correspondant au réseau du serveur web légitime.
- **AS1 (S1)** agit comme routeur de transit principal, recevant les annonces BGP et les redistribuant vers les autres AS.
- **AS2 (S2)** reçoit les annonces de routage via AS1 et choisit ses routes en fonction des informations BGP reçues.
- **AS4 (S4)** est présent dans la topologie mais n'annonce aucune route malveillante dans ce scénario initial.

3.3 Mise en place du routage BGP normal

Au démarrage de la topologie, chaque routeur exécute les démons **zebra** et **bgpd** de **FRRouting**, assurant respectivement la gestion de la table de routage et l'échange des annonces BGP.

Le lancement de la topologie s'effectue à l'aide de la commande suivante :

```
sudo python3 bgp.py
```

Une fois la topologie initialisée, les sessions BGP sont établies entre les routeurs voisins, et les annonces de préfixes sont échangées automatiquement.

3.4 Annonces BGP dans l'état normal

Dans l'état normal, le préfixe `13.0.0.0/8` est annoncé par **AS3** et propagé progressivement aux autres systèmes autonomes.
Chaque routeur enrichit l'attribut **AS_PATH** avant de relayer l'annonce à ses voisins.

Par exemple :

- AS3 annonce `13.0.0.0/8` ;
- AS2 reçoit l'annonce avec un AS_PATH contenant AS3 ;
- AS1 reçoit ensuite l'annonce via AS2, avec un AS_PATH plus long.

Ces annonces peuvent être observées directement sur les routeurs à l'aide de la commande suivante :

```
sh ip bgp
```

```
bgpd-S1> sh ip bgp
BGP table version is 3, local router ID is 9.0.0.1, vrf id 0
Default local pref 100, local AS 1
Status codes:  s suppressed, d damped, h history, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes:  i - IGP, e - EGP, ? - incomplete

   Network          Next Hop            Metric LocPrf Weight Path
*> 11.0.0.0/8       0.0.0.0                  0         32768 i
*> 12.0.0.0/8       9.0.0.2                  0             0 2 i
*> 13.0.0.0/8       9.0.0.2                                 0 2 3 i

Displayed  3 routes and 3 total paths
```

3.5 Validation applicative du routage normal

Afin de valider le bon fonctionnement du routage BGP au niveau applicatif, des requêtes HTTP sont effectuées vers le serveur web légitime.

La commande suivante permet de vérifier la réponse du serveur :

```
curl 13.0.1.1
```

Dans l'état normal, la réponse provient du **serveur légitime**, confirmant que le trafic est correctement acheminé vers AS3.

```
mininet@mininet-vm:~/PROJET_FINAL$ sudo python3 run.py --node h2-1 --cmd "curl 13.0.1.
1"
Executed cmd: mnexec -a 983 curl 13.0.1.1
<h1>Default web server</h1>
mininet@mininet-vm:~/PROJET_FINAL$
```

4. Attaque de BGP Hijacking par annonce du même préfixe avec un AS_PATH plus court

4.1 Principe de l'attaque

La première attaque étudiée repose sur un mécanisme classique de **BGP Hijacking par AS_PATH plus court**.
Dans ce scénario, le routeur attaquant annonce **le même préfixe IP que le réseau légitime**, mais avec un **chemin BGP artificiellement plus attractif**, c'est-à-dire un **AS_PATH plus court**.

Dans la topologie mise en place :

- le réseau légitime (AS3) annonce le préfixe `13.0.0.0/8` ;
- le routeur attaquant (AS4) annonce également `13.0.0.0/8`, mais directement vers AS1.

En raison du fonctionnement interne de BGP, un chemin comportant moins de systèmes autonomes est préféré lors du processus de sélection de route.

4.2 Lancement de l'attaque

L'attaque est déclenchée à l'aide d'un script dédié, permettant de démarrer les démons BGP du routeur attaquant avec une configuration spécifique :

```
./start_rogue.sh
```

Ce script lance le démon **bgpd** sur AS4 avec une annonce du préfixe `13.0.0.0/8` vers AS1, en contournant le chemin légitime plus long.

4.3 Observation des annonces BGP

Une fois l'attaque lancée, les annonces BGP peuvent être observées sur les routeurs à l'aide de la commande :

```
show ip bgp
```

Sur **AS1 (S1)**, deux routes vers le même préfixe `13.0.0.0/8` sont visibles :

- la route légitime via AS2 et AS3 ;
- la route malveillante via AS4, avec un AS_PATH plus court.

BGP sélectionne alors la route annoncée par AS4, marquée par le symbole `*>`, indiquant qu'elle est **préférée et installée dans la table de routage**.

```
bgpd-S1> sh ip bgp
BGP table version is 4, local router ID is 9.0.0.1, vrf id 0
Default local pref 100, local AS 1
Status codes:  s suppressed, d damped, h history, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes:  i - IGP, e - EGP, ? - incomplete

   Network          Next Hop            Metric LocPrf Weight Path
*> 11.0.0.0/8       0.0.0.0                  0         32768 i
*> 12.0.0.0/8       9.0.0.2                  0             0 2 i
*> 13.0.0.0/8       9.0.4.2                  0             0 4 i
*                   9.0.0.2                                 0 2 3 i
```

4.4 Visibilité de l'attaque depuis AS2

Il est important de noter que l'attaque est également **visible depuis AS2 (S2)**, mais son impact sur la sélection de route diffère.

Sur AS2, la commande :

```
show ip bgp
```

montre également deux routes vers le préfixe `13.0.0.0/8` :

- une route légitime vers AS3 ;
- une route propagée depuis AS1, correspondant à l'annonce malveillante d'AS4.

Cependant, contrairement à AS1, **AS2 dispose déjà d'une route plus courte vers le préfixe légitime**, via son voisin direct AS3.
Ainsi, la route légitime reste préférée et est marquée par `*>`, tandis que la route issue de l'attaque apparaît uniquement avec le symbole `*`, indiquant qu'elle est connue mais **non sélectionnée**.

```
bgpd-S2> sh ip bgp
BGP table version is 3, local router ID is 9.0.0.2, vrf id 0
Default local pref 100, local AS 2
Status codes:  s suppressed, d damped, h history, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes:  i - IGP, e - EGP, ? - incomplete

   Network          Next Hop            Metric LocPrf Weight Path
*> 11.0.0.0/8       9.0.0.1                  0             0 1 i
*> 12.0.0.0/8       0.0.0.0                  0         32768 i
*  13.0.0.0/8       9.0.0.1                                 0 1 4 i
*>                  9.0.1.2                  0             0 3 i
```

4.5 Impact applicatif de l'attaque

Malgré le fait qu'AS2 conserve une route légitime, le changement de routage sur AS1 suffit à détourner le trafic provenant des hôtes connectés à ce routeur.

Une requête HTTP vers le serveur web permet d'observer l'impact applicatif direct de l'attaque :

```
curl 13.0.1.1
```

La réponse HTTP provient alors du **serveur attaquant**, démontrant que le trafic a bien été redirigé malgré l'existence de routes légitimes ailleurs dans la topologie.

```
Wed 21 Jan 2026 10:45:16 AM PST -- <h1>*** Attacker web server ***</h1> Executed cmd:
mnexec -a 977 curl -s 13.0.1.1
```

4.6 Analyse et limites de l'attaque

Cette attaque illustre un point important du fonctionnement de BGP :
la sélection de route est **locale à chaque système autonome**.
Ainsi, un routeur peut être affecté par une attaque tandis qu'un autre, disposant d'un chemin plus court vers la destination légitime, ne modifie pas sa route.

Dans ce scénario :

- AS1 est directement impacté par l'attaque ;
- AS2 observe l'annonce malveillante mais ne la sélectionne pas ;
- le trafic applicatif est néanmoins détourné à partir des zones affectées.

4.7 Conclusion sur l'attaque par AS_PATH plus court

L'attaque par annonce du même préfixe avec un AS_PATH plus court démontre la fragilité du modèle de confiance de BGP.
Elle met en évidence la facilité avec laquelle un routeur peut être trompé et rediriger du trafic vers un AS malveillant, sans mécanisme de vérification de l'authenticité des annonces.

Cette attaque constitue une base de comparaison essentielle pour la partie suivante, qui étudie une attaque plus subtile et souvent plus dangereuse : l'annonce d'un **préfixe plus spécifique**.

5. Attaque de BGP Hijacking par annonce d'un préfixe plus spécifique

5.1 Principe et motivation de l'attaque

La seconde attaque étudiée repose sur un mécanisme fondamental du routage IP : le **Longest Prefix Match**.
Contrairement à l'attaque par AS_PATH plus court, cette attaque ne cherche pas à rendre le chemin plus attractif en termes de longueur, mais à **annoncer un préfixe plus précis** que celui du réseau légitime.

Dans ce scénario :

- le réseau légitime annonce le préfixe global `13.0.0.0/8` ;
- le routeur attaquant annonce un sous-préfixe `13.0.1.0/24`.

Selon les règles du routage IP et de BGP, **une route plus spécifique est toujours prioritaire**, indépendamment de la longueur du chemin ou du nombre de systèmes autonomes traversés.

5.2 Lancement de l'attaque

L'attaque est déclenchée en démarrant le routeur attaquant avec une configuration BGP modifiée annonçant le préfixe plus spécifique :

```
./start_rogue_specific.sh
```

Cette annonce est ensuite propagée dans l'ensemble de la topologie via les sessions BGP existantes.

5.3 Observation des annonces BGP

Après le lancement de l'attaque, les tables BGP des routeurs peuvent être analysées à l'aide de la commande :

```
show ip bgp
```

Sur **AS1 (S1)**, la route `13.0.1.0/24` annoncée par AS4 est sélectionnée et marquée par `*>`, remplaçant implicitement la route plus générale `13.0.0.0/8`.

```
bgpd-S1> sh ip bgp
BGP table version is 6, local router ID is 9.0.0.1, vrf id 0
Default local pref 100, local AS 1
Status codes:  s suppressed, d damped, h history, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes:  i - IGP, e - EGP, ? - incomplete

   Network          Next Hop            Metric LocPrf Weight Path
*> 11.0.0.0/8       0.0.0.0                  0         32768 i
*> 12.0.0.0/8       9.0.0.2                  0             0 2 i
*> 13.0.0.0/8       9.0.0.2                                 0 2 3 i
*> 13.0.1.0/24      9.0.4.2                  0             0 4 i
```

5.4 Impact sur AS2 : un cas particulièrement révélateur

L'aspect le plus significatif de cette attaque apparaît lors de l'analyse du comportement de **AS2 (S2)**.

Contrairement au scénario précédent, **AS2 est directement connecté au réseau légitime AS3** et dispose donc, en situation normale, d'un chemin plus court et direct vers le serveur légitime.

Cependant, après l'annonce du préfixe plus spécifique, AS2 modifie lui aussi sa décision de routage. Cette décision peut être observée à l'aide de la commande suivante :

```
ip route get 13.0.1.1
```

La sortie obtenue est la suivante :

```
13.0.1.1 via 9.0.0.1 dev S2-eth4 src 9.0.0.2
```

Cette information montre clairement que **AS2 choisit désormais un chemin passant par AS1**, puis par le routeur attaquant, **malgré l'existence d'un chemin plus court vers le réseau légitime**.

```
bgpd-S2> sh ip bgp
BGP table version is 4, local router ID is 9.0.0.2, vrf id 0
Default local pref 100, local AS 2
Status codes:  s suppressed, d damped, h history, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes:  i - IGP, e - EGP, ? - incomplete

   Network          Next Hop            Metric LocPrf Weight Path
*> 11.0.0.0/8       9.0.0.1                  0             0 1 i
*> 12.0.0.0/8       0.0.0.0                  0         32768 i
*> 13.0.0.0/8       9.0.1.2                  0             0 3 i
*> 13.0.1.0/24      9.0.0.1                                 0 1 4 i
```

5.5 Analyse du changement de décision de routage

Ce comportement s'explique par le fait que le processus de sélection de route privilégie d'abord la **spécificité du préfixe**, avant tout autre critère, y compris :

- la longueur de l'AS_PATH ;
- la proximité topologique ;
- le nombre de sauts réseau.

Ainsi, même si AS2 est voisin direct du réseau légitime, la présence d'un préfixe `/24` entraîne une redirection systématique du trafic vers l'AS attaquant.

Cette caractéristique rend ce type d'attaque **particulièrement puissant et difficile à contrer**.

5.6 Impact applicatif de l'attaque

Comme dans le scénario précédent, l'impact de cette attaque est directement observable au niveau applicatif.

Une requête HTTP effectuée vers l'adresse ciblée renvoie désormais une réponse provenant du **serveur attaquant**, confirmant que le trafic est effectivement détourné :

```
curl 13.0.1.1
```

```
mininet@mininet-vm:~/PROJET_FINAL$ sudo python3 run.py --node h2-1 --cmd "curl 13.0.1.
1"
Executed cmd: mnexec -a 983 curl 13.0.1.1
<h1>*** Attacker web server ***</h1>
```

(La requête est faite depuis R2 qui va quand meme passer par R1 et R4 et atterrir sur le serveur attaquant)

5.7 Comparaison avec l'attaque par AS_PATH plus court

Cette attaque met en évidence une différence majeure avec l'attaque par AS_PATH plus court :

- l'attaque par AS_PATH plus court peut être limitée à certains routeurs ;
- l'attaque par préfixe plus spécifique affecte **l'ensemble des systèmes autonomes**, même ceux disposant d'un chemin direct vers la destination légitime.

Elle est donc **plus globale**, **plus discrète** et **plus dangereuse**.

5.8 Conclusion sur l'attaque par préfixe plus spécifique

L'attaque par annonce d'un préfixe plus spécifique illustre l'une des faiblesses les plus critiques de BGP.
Elle démontre que la simple annonce d'un sous-réseau suffit à détourner le trafic, sans nécessiter de manipulation complexe du chemin ou des politiques de routage.

Ce scénario met en évidence la nécessité de mécanismes de protection supplémentaires, tels que le filtrage de préfixes ou la validation cryptographique des annonces BGP, afin de limiter les risques de détournement à grande échelle.

Conclusion générale

Ce projet avait pour objectif d'étudier le fonctionnement du protocole **Border Gateway Protocol (BGP)** et de mettre en évidence ses vulnérabilités à travers la mise en œuvre d'attaques de type **BGP Hijacking** dans un environnement contrôlé.

La première étape du travail a consisté à mettre en place l'environnement expérimental à l'aide de **Mininet** et de **FRRouting**, en adaptant un code existant initialement développé pour des versions obsolètes de Python et de certaines dépendances réseau. Cette phase de mise en place a nécessité un travail conséquent de débogage, de mise à jour et de compréhension approfondie du fonctionnement interne des outils utilisés. Elle a permis de disposer d'une topologie stable et reproductible, indispensable à l'analyse des attaques.

Dans un second temps, le fonctionnement normal de BGP a été étudié afin d'établir une référence claire. Cette étape a permis d'observer la propagation des annonces légitimes, la construction des attributs BGP, ainsi que la sélection des routes en l'absence de toute attaque. Elle a également mis en évidence le caractère local de la décision de routage, propre à chaque système autonome.

Deux attaques de BGP Hijacking distinctes ont ensuite été mises en œuvre et analysées.
La première attaque, basée sur l'annonce du même préfixe avec un **AS_PATH plus court**, a montré qu'un routeur peut être trompé en privilégiant un chemin artificiellement plus attractif. Cette attaque a mis en évidence le fait que certaines parties du réseau peuvent être impactées tandis que d'autres, disposant d'un chemin plus court vers la destination légitime, ne modifient pas leur décision de routage.

La seconde attaque, reposant sur l'annonce d'un **préfixe plus spécifique**, s'est révélée encore plus pertinente et dangereuse. En exploitant la règle du **Longest Prefix Match**, cette attaque a permis de détourner le trafic même depuis des systèmes autonomes directement connectés au réseau

légitime. Les résultats expérimentaux ont montré que la spécificité du préfixe prime sur la longueur du chemin, entraînant une redirection systématique du trafic vers l'AS attaquant.

L'impact de ces attaques a été observé à plusieurs niveaux : au niveau du contrôle-plane (annonces BGP), du data-plane (tables de routage) et du niveau applicatif, à travers le détournement effectif des requêtes HTTP. Ces observations confirment que les vulnérabilités de BGP ne sont pas uniquement théoriques, mais peuvent avoir des conséquences directes et transparentes pour les utilisateurs finaux.

Ce projet met ainsi en évidence les limites du modèle de confiance sur lequel repose BGP et souligne l'importance de mécanismes de sécurisation complémentaires, tels que le filtrage de préfixes, les politiques de routage strictes ou encore les solutions de validation cryptographique comme **RPKI**. Bien que ces mécanismes ne soient pas encore déployés de manière universelle, leur adoption apparaît essentielle pour renforcer la résilience du routage inter-domaine sur Internet.